COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE010**	**SECTION:**	**CPE21S2**
DATE PERFORMED:	**July 17, 2026**	**DATE SUBMITTED:**	**July 17, 2026**
NAME:	**Macatangay, Justin Jake B.**	**INSTRUCTOR:**	**Engr. Jimlord Quejado**

ACTIVITY NO. 2.1 [Arrays, Pointers and Dynamic Memory Allocation]

1. PROCEDURE OUTPUT

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:

      //Constructor
      Student(std::string newName ="Joe Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
      };

      //Deconstructor
      ~Student(){
        std::cout << "Destrcutor Called." << std::endl;
      }

      //Copy Constructor
      Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
      }

      //Display Attributes
      void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;

      }

};
```

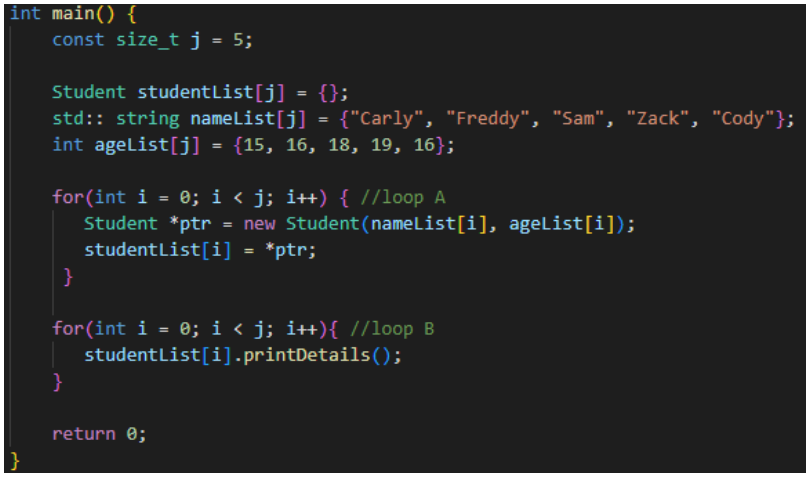
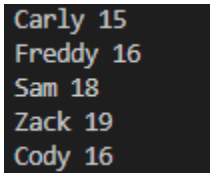
```
Constructor Called.
Copy Constructor Called
Constructor Called.
Destrcutor Called.
Destrcutor Called.
Destrcutor Called.
```

Observation

In the first driver program, the visualization from the Python Tutor tool clearly demonstrated how one instance of a Student object is created through a constructor call and then destroyed

	when the program is executed to the end. The call sequence of the constructor and the destructor proved that the correct object life cycle works.

Table 2-1. Initial Driver Program

Screenshot	 
Observation	In the case of Loop A, a new Student object was created dynamically during every execution of the loop, resulting in the calling of constructors as the process of iterating through the names and ages was performed. Once the information is assigned to the array, the destructor gets called during the deletion of the dynamically allocated memory. On the other hand, Loop B successfully prints out all students' information, indicating that the information that has been stored in the array has been retrieved properly.

```
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std:: string nameList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

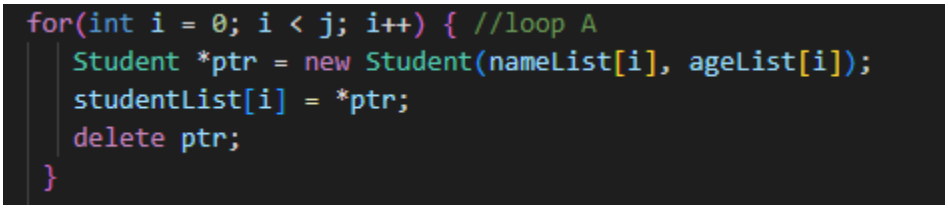
    for(int i = 0; i < j; i++) { //loop A
       Student *ptr = new Student(nameList[i], ageList[i]);
       studentList[i] = *ptr;
     }

    for(int i = 0; i < j; i++){ //loop B
       studentList[i].printDetails();
    }

    return 0;
}
```

```
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
```

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	The visualization by the Python Tutor for the last driver program showed the flawless running of the program after making the modifications. All objects were made use of the constructor, while the copy constructor was used when an attempt to duplicate existed objects was done. The destructor functions were sequentially called at the end of the program, showing that all the dynamically allocated memory was freed.

```
for(int i = 0; i < j; i++) { //loop A
    Student *ptr = new Student(nameList[i], ageList[i]);
    studentList[i] = *ptr;
    delete ptr;
  }
```

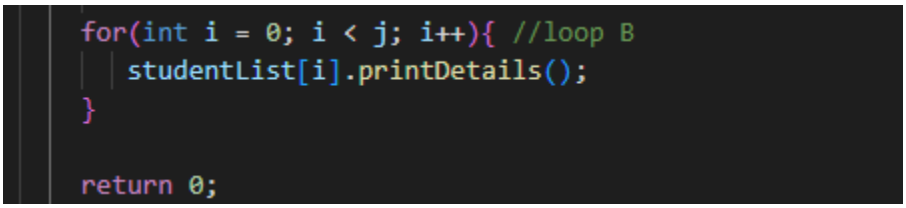
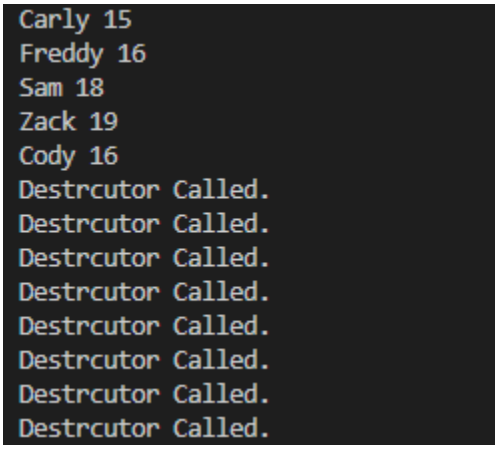
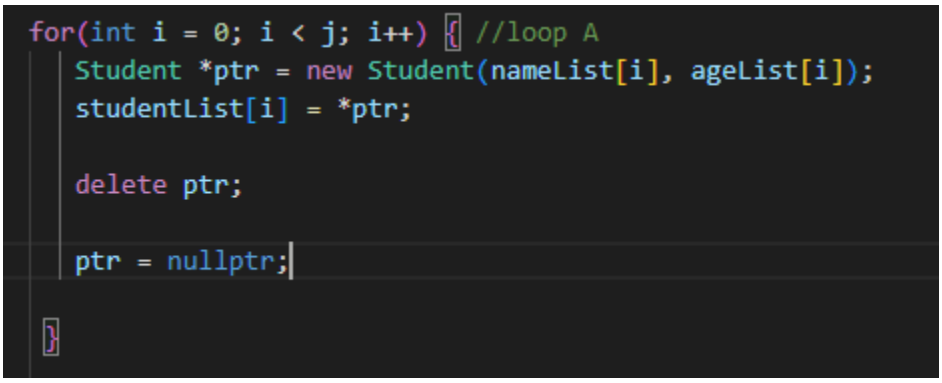
Loop B	
Observation	In the second case, the program provided the object lifetimes that were constant across various students' input data. The constructor created the students' names and ages, whereas the copy constructor duplicated them correctly. The destructor was called automatically when the lifetime of an object came to an end, and hence all objects were destroyed automatically. Thus, the program provided the results that corresponded to the initial purpose of the programming.
Output	
Observation	The final code for the driver was found to be well-defined in terms of execution. The constructors were correctly initializing all data of students, while the destructors were running properly when the objects were out of their scope. It was clear from the visualization that the logic of the code with regard to arrays, pointers, and dynamic memory allocation was working well.

Table 2-3. Final Driver Program

Modification	

Observation	The required changes included the deletion of redundant declaration of studentList, nameList, and ageList and the inclusion of specific delete statements for the release of dynamically allocated memory. This allowed the program to avoid having redeclaration issues and to prevent any occurrence of memory leaks. It was observed that there were accurate instances of object construction and destruction with the proper display of constructors and destructors in the visualized program.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.